Cybersecurity & Trust Layer

Development, Verification, Testing and Acceptance Guide for the Battery Thermal Intelligence Cloud Platform

Scope: Step 3 of the supplied Battery Thermal Intelligence Cloud Platform architecture: AI-based Cyber IDS, physics-based plausibility validation, anomaly/spoofing detection, PKI/key management, SIEM/audit monitoring, encryption, incident response, SAFE-versus-SPOOFED decision logic, and attack-evidence anchoring.

Purpose: Provide a practical implementation workflow that engineering teams can follow from requirements through development, security validation, acceptance, deployment, and field monitoring.

Important: The architecture blocks and terminology are derived from the supplied platform figure. The detailed development lifecycle, test methods, acceptance targets and standards mapping in this guide are engineering recommendations added to make the architecture executable.

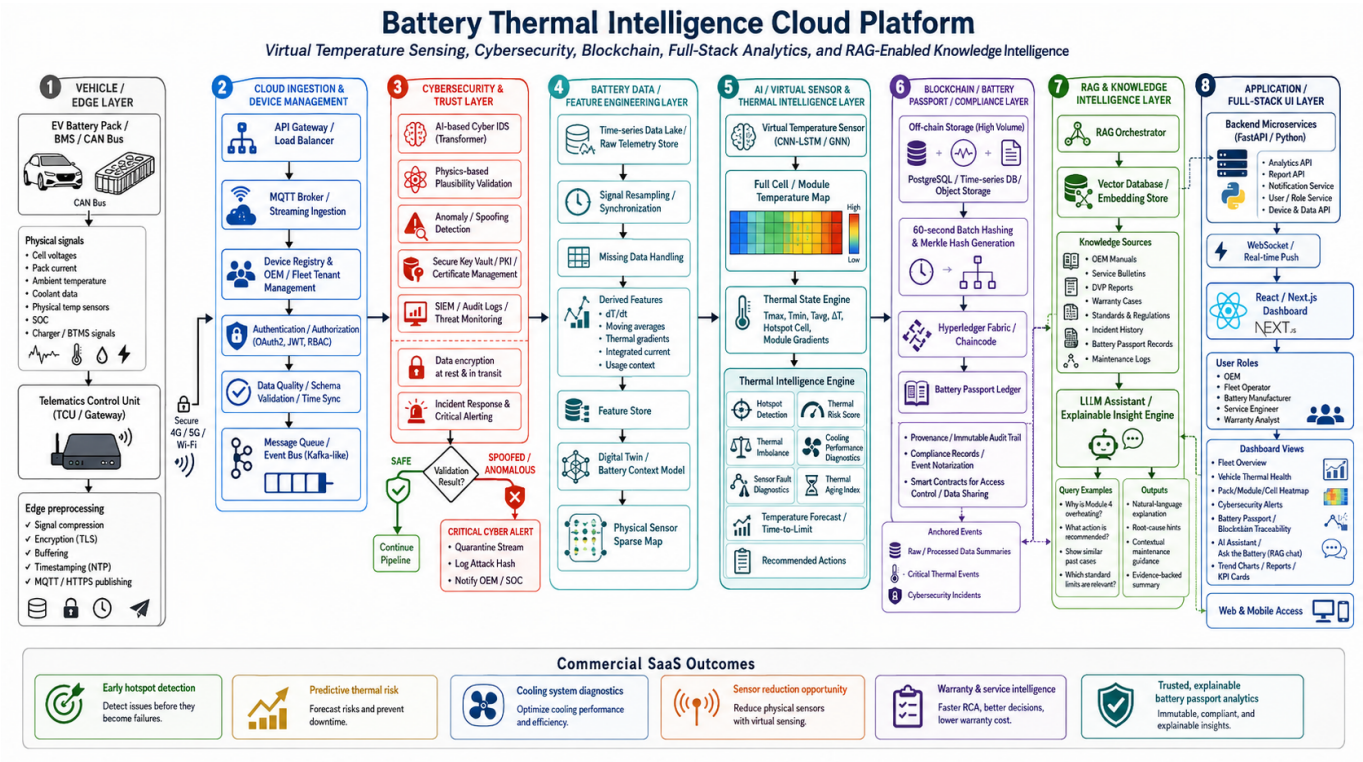
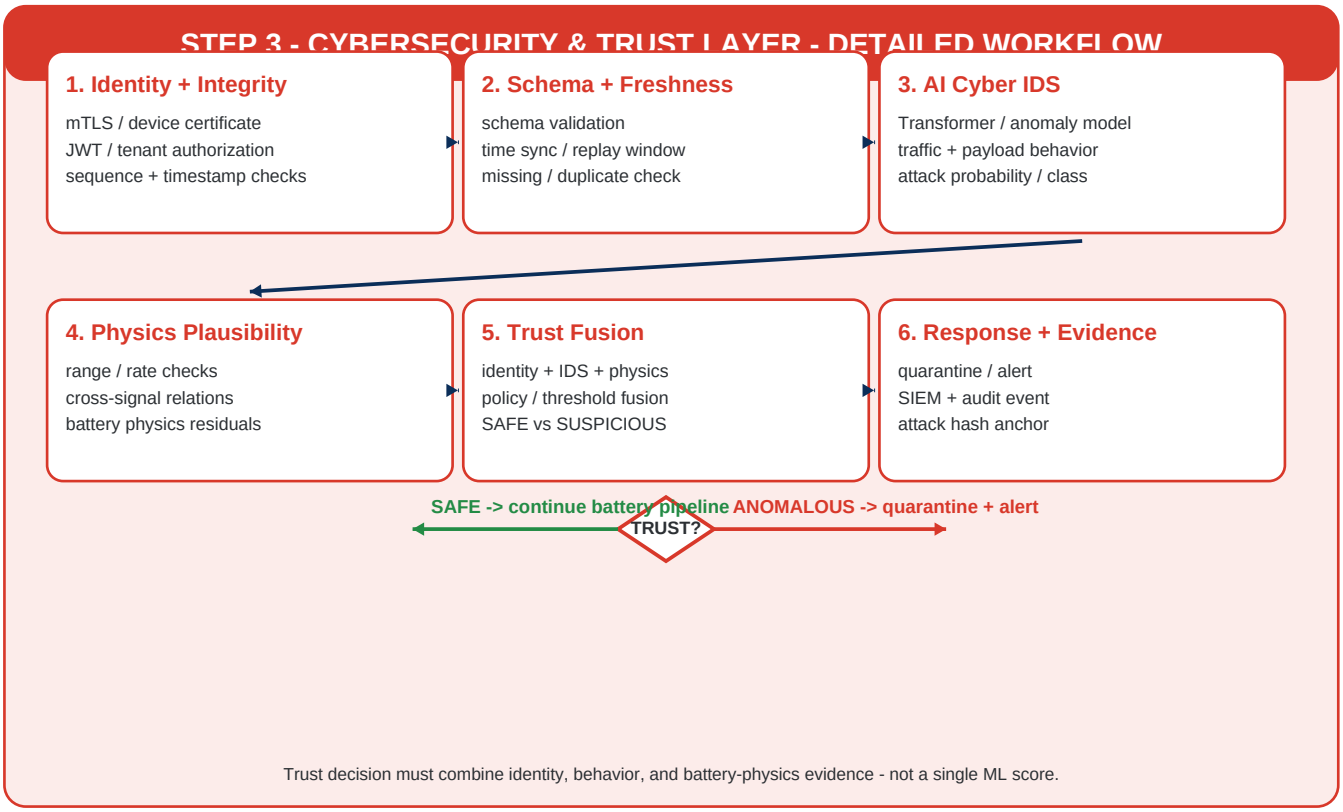


Figure 1. Source architecture supplied for this guide. Step 3 is the Cybersecurity & Trust Layer.

1. What Step 3 Must Achieve

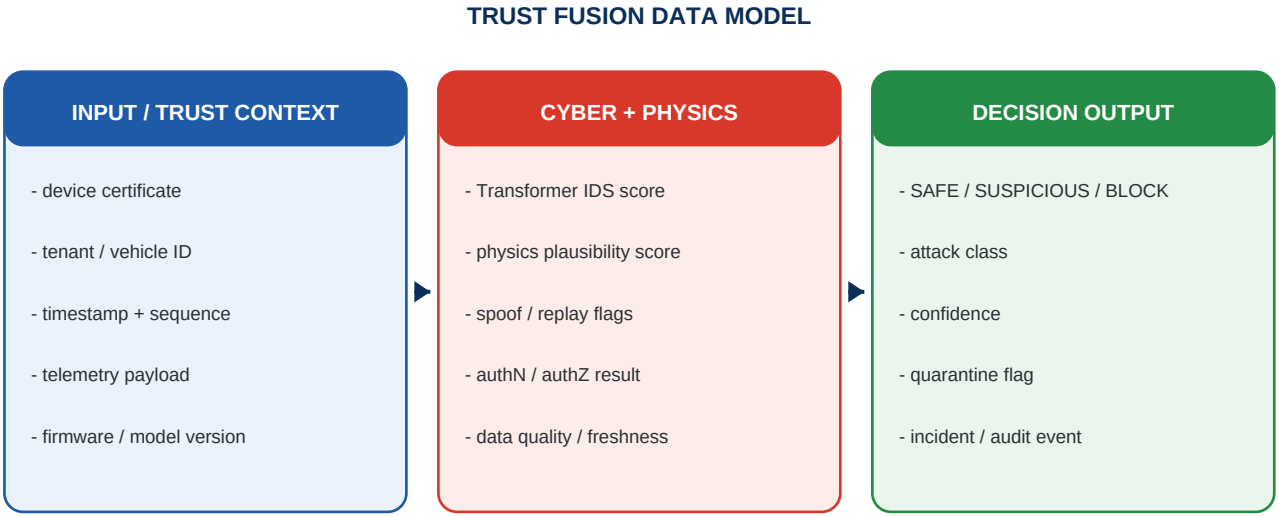
The Cybersecurity & Trust Layer is not only an intrusion detector. It is the decision boundary that determines whether incoming battery telemetry is sufficiently trusted to enter the downstream feature engineering, virtual sensing, thermal intelligence, blockchain, RAG and customer-facing analytics pipeline.



Security objective	What the layer must decide	Primary evidence
Authenticity	Did the data originate from an authorized TCU/ device?	mTLS certificate, device identity, token, tenant binding
Integrity	Was the payload altered, duplicated or replayed?	TLS, sequence number, timestamp, signature/ hash, replay window
Behavioral normality	Does the traffic/ payload behavior resemble the known fleet/ device baseline?	Transformer/ IDS score, rate/ frequency patterns, payload patterns
Physical plausibility	Can these battery signals coexist physically?	range/ rate/ cross-signal rules, SOC-current-voltage-temperature relations
Operational trust	Can downstream AI use this sample?	fusion policy: SAFE, SUSPICIOUS, BLOCK
Forensic traceability	Can the event be reconstructed later?	SIEM log, immutable event ID, model/ rule version, hash anchor

2. Functional Scope and Interfaces

The trust layer sits between cloud ingestion/device management and the battery data/feature engineering layer. It consumes telemetry plus security context and produces both a trusted data stream and security evidence.



Recommended input data contract

Field family	Examples	Why needed
Identity	tenant_id, vehicle_id, pack_id, tcu_id, certificate subject	Bind telemetry to an authorized asset and tenant
Timing	event_time, ingest_time, sequence_number	Freshness, replay detection, ordering and latency
Telemetry	cell/ module voltage, pack current, SOC, temperatures, coolant/ BTMS states	Physics and behavioral checks
Quality	missing flags, stale flags, source quality, decode status	Prevent low-quality data from being trusted
Platform context	firmware_version, DBC/ schema_version, IDS_model_version	Traceability and rollout control
Security context	mTLS result, token claims, source IP/ network zone	Authentication and authorization evidence

Output contract: trusted_payload, trust_status, cyber_score, physics_score, attack_class, confidence, reason_codes, quarantine_flag, incident_id, audit_hash and model/rule versions.

3. Detailed Runtime Workflow

3.1 Device and tenant authentication

Perform mutual TLS or equivalent device authentication. Validate certificate chain, expiry, revocation, tenant/device binding and authorization policy. Reject or quarantine unknown identities before model inference.

3.2 Message integrity and freshness

Validate schema, timestamp, sequence number, payload length, decoder status, duplicate/replay window and expected publishing cadence. Mark out-of-order or stale samples explicitly.

3.3 AI-based Cyber IDS

Run the Transformer/sequence-based IDS on traffic and payload behavior. Typical features include message frequency, inter-arrival statistics, ID/signal sequence, entropy, payload bit/signal changes, device context and historical behavior.

3.4 Physics-based plausibility validation

Apply deterministic and learned physics checks: absolute ranges, rate-of-change, charge/discharge sign consistency, voltage-current-power consistency, SOC evolution, thermal slew limits, cooling response plausibility and cross-sensor consistency.

3.5 Anomaly and spoofing correlation

Correlate identity, timing, IDS and physics outputs. A payload can be cryptographically valid yet physically impossible; conversely, a legitimate battery fault can look statistically unusual but must not automatically be classified as a cyberattack.

3.6 Trust fusion and policy decision

Generate SAFE, SUSPICIOUS or BLOCK. Use policy logic with explainable reason codes. Do not rely on one ML confidence score as the sole gate.

3.7 SAFE path

Forward trusted payload to the battery data/feature engineering layer. Attach trust metadata so downstream virtual-sensor models can use data-quality and confidence information.

3.8 SUSPICIOUS/BLOCK path

Quarantine the sample or stream, generate a critical cyber alert when required, preserve evidence, prevent the untrusted data from influencing safety/thermal intelligence, and notify the OEM/SOC according to severity.

3.9 Evidence and blockchain anchoring

Write the detailed event off-chain; generate a SHA-256/Merkle summary and anchor only the needed digest/event proof to Hyperledger. Do not place high-volume or sensitive raw telemetry on-chain.

3.10 Continuous monitoring and learning

Monitor false positives, missed attacks, model drift, new fleet/device behavior, rule hit rates, certificate failures and incident trends. Update rules/models through a controlled versioned release process.

4. Physics-Based Plausibility Layer

Physics validation is the key differentiator between a generic cloud IDS and a battery-aware trust layer. It reduces false positives and helps distinguish cyber manipulation from genuine electro-thermal faults.

Check family	Example rule	Expected output
Range	Vcell_min <= Vcell_i <= Vcell_max; SOC 0-100%; configured temperature range	PASS / FAIL + signal ID
Rate-of-change	abs(dV/ dt), abs(dT/ dt), abs(dSOC/ dt) below calibrated physical bounds	PASS / FAIL + magnitude
Current/ SOC consistenc y	Discharge current should not increase SOC beyond tolerance; charging should not reduce SOC unexpectedly	Residual / inconsistency score
Power consistency	Pack power approx pack_voltage * pack_current within tolerance	Residual and confidence
Cell topology	Cell voltages should belong to expected module/ channel count and ordering	Topology / schema anomaly
Thermal consistency	Local sensor deviation versus neighboring/ virtual temperature should remain within learned/ calibrated envelope	Thermal residual
BTMS response	High cooling command should produce plausible thermal response after calibrated delay	Cooling plausibility score
Timestamp/ cadence	Event age and inter-arrival time within configured publish window	fresh / stale / replay-suspect

Calibration note: numerical battery limits are product-specific. Acceptance should use OEM-defined electrical and thermal envelopes rather than hard-coded generic thresholds.

5. AI-Based Cyber IDS Development

The Transformer/sequence IDS should be developed as a supervised, semi-supervised or hybrid detector. A practical first release should combine learned anomaly scoring with deterministic security and physics rules.

Development activity	Recommended method	Evidence produced
Dataset definition	Collect clean fleet/ bench telemetry plus controlled cyber-attack injections	versioned dataset manifest and labels
Feature pipeline	traffic timing + payload/ signal sequences + identity/ context features	feature specification and reproducible code
Train/ validation/ test split	split by vehicle/ device/ session/ attack campaign, not random adjacent packets	no-leakage split report
Model training	Transformer or sequence encoder, calibrated probability/ score	model artifact + training log
Threshold selection	ROC/ PR curve plus cost-weighted false-positive analysis	selected operating point
Explainability	reason codes, attention/ feature contribution where feasible	analyst-facing explanation
Model governance	model card, data lineage, model hash, version, rollback path	release package

Minimum model metrics should include precision, recall, F1, false-positive rate, false-negative rate, detection latency, per-attack-class confusion matrix and performance under clean unseen vehicles.

6. Attack and Fault Injection Catalogue

The validation campaign must contain both cyberattacks and genuine battery/sensor faults. Otherwise the IDS may incorrectly flag real battery problems as cyber events.

ID	Scenario	Injection method	Expected classification / action
A01	Replay attack	Replay valid historical payload with old sequence/ timestamp	replay-suspect -> quarantine
A02	Signal spoofing	Bias cell voltage / temperature / SOC while preserving syntax	physics/ IDS anomaly -> quarantine or suspicious
A03	Message injection	Inject extra high-rate telemetry frames/ events	rate anomaly -> alert
A04	Timestamp manipulation	future/ old timestamps, reordered sequence	freshness violation
A05	Payload tampering	modify message after edge source / invalid integrity context	integrity/ security failure
A06	Credential misuse	valid but unauthorized tenant/ device/ token	authorization failure -> block
A07	Expired/ revoked cert	use revoked device certificate	authentication failure -> block
A08	Cross-tenant access	attempt to read/ write another OEM/ fleet tenant	RBAC/ ABAC deny + audit
A09	API abuse / BOLA	object-ID manipulation against backend APIs	deny + API security alert
A10	Flood / DoS	burst ingestion above provisioned rate	rate limit, protect service, alert
A11	Data poisoning attempt	inject manipulated training/ feedback labels	dataset governance / quarantine
A12	Model evasion	crafted values near decision boundary	evaluate robustness + reason codes
F01	Real sensor stuck	temperature sensor freezes while pack changes	battery fault/ sensor fault, not cyber by default
F02	Cooling degradation	pump/ fan effectiveness reduced	physical anomaly; downstream thermal diagnostic
F03	Actual cell hotspot	local temperature rise with valid identity	physical fault; SAFE data + thermal alert
F04	Missing/ late telemetry	network delay/ dropout	data-quality degraded; not automatically cyber

7. PKI, Key Management and Data Protection

Control area	Development requirement	Acceptance method
Device identity	Unique device certificate and key protected by TCU secure storage when available	positive/ negative mTLS tests; duplicate identity test
Certificate lifecycle	issue, renew, rotate, revoke, expiry handling	automated lifecycle test + revoked-cert block test
Secrets	no hard-coded production secrets in source/ images	secret scan + code review
Transport	TLS 1.2+; prefer TLS 1.3 where ecosystem allows	protocol scan / packet inspection
At-rest protection	cloud KMS managed encryption for DB/ object storage/ backups	configuration evidence + restore test
Authorization	tenant isolation and least privilege roles	cross-tenant negative test / RBAC matrix
Audit	administrative, security and key operations logged	audit completeness test

Recommended implementation principle: raw private battery telemetry remains off-chain. Blockchain stores cryptographic proof, event references and controlled compliance records, not unrestricted raw payloads.

8. SIEM, Audit Logs and Incident Response

Every security decision must be reconstructable. The SOC should be able to answer: what happened, to which asset, which rule/model triggered, what data was quarantined, who was notified, and whether the evidence hash still verifies.

Event field	Required content
Identity	tenant, vehicle, pack, TCU, certificate/ device ID
Timing	event time, ingestion time, detection time, alert time
Decision	SAFE / SUSPICIOUS / BLOCK, severity, reason codes
Scores	IDS score, physics score, trust score, threshold/ version
Versions	schema, rule set, IDS model, firmware if known
Evidence	payload/ event hash, source reference, quarantine location
Response	alert recipients, ticket ID, block/ quarantine action, operator acknowledgement

Incident playbook

- Detect and classify the event.
- Contain: quarantine stream, revoke/disable credentials if necessary, rate-limit or isolate source.
- Preserve evidence and calculate immutable event hash.
- Notify OEM/SOC according to severity and tenant policy.
- Investigate cyber versus physical-fault hypothesis using raw + physics context.
- Recover service using known-good identity/model/configuration.
- Perform post-incident review and update rules/model/test cases.

9. Development Lifecycle and Completion Timeline

The development lifecycle below converts the trust-layer architecture into an executable engineering program. Durations are planning targets for a prototype-to-acceptance implementation and should be tailored to team size, data availability, OEM interfaces and security-assurance scope.

Phase	Main activities	Target duration / planned completion	Key deliverables	Exit condition
P0 - Scope & Assets	data-flow diagram, asset inventory, tenant boundaries, attack surfaces	0.5 week - complete in Week 1	Cybersecurity Plan, asset list, interface map	scope approved
P1 - Threat Analysis	threat scenarios, attack paths, risk ranking, trust assumptions	0.5 week - complete in Week 1	TARA/ threat model, security goals	security concept approved
P2 - Requirements	derive functional, security, logging and performance requirements	0.5 week - complete by mid-Week 2	Cybersecurity Requirements Specification	requirements baselined
P3 - Architecture	trust pipeline, PKI, IDS, physics engine, SIEM, quarantine, blockchain evidence	1 week - complete by end-Week 2 / early Week 3	Cyber Architecture + interface/ data contracts	architecture review passed
P4 - Component Build	auth, schema, rules, IDS model, key vault, audit, incident service	4 weeks - Weeks 3-6	unit-tested services + model artifact	unit coverage / static checks passed
P5 - Integration	connect ingestion -> trust -> data pipeline, safe/ anomaly branches	1 week - Week 7	integrated environment + replay harness	interface tests passed
P6 - Verification	attack injection, fault injection, performance, pen test, model metrics	2 weeks - Weeks 8-9	DVP, test reports, vulnerability report	acceptance criteria met
P7 - Pilot	shadow mode on fleet/ bench, SOC workflow validation, threshold tuning	1.5 weeks - Weeks 10-11	pilot report, tuned thresholds	pilot exit gate
P8 - Release	security sign-off, model/ rule freeze, SBOM, runbooks, rollback	1 week - Week 12	production release package	production readiness review

Planned program duration: approximately 12 weeks for a prototype-to-acceptance baseline. Production programs may require additional time for OEM cybersecurity governance, independent penetration testing, infrastructure hardening, data-privacy review and regulatory evidence.

9.1 Minimum Validation Evidence

- **Baseline reproduction:** reproduce at least one published IDS baseline before tuning the proposed IDS.
- **Dataset separation:** keep independent attack sessions / vehicles for test; document every split and random seed.
- **Attack coverage:** replay, fabrication/spoofing, fuzzing, flooding/DoS, suppression, stale timestamp and masquerade where the selected dataset supports them.
- **Metrics:** precision, recall, F1, false-positive rate, false-negative rate, ROC/PR, confusion matrix and p95 inference/trust-decision latency.
- **Physics cross-check:** include genuine battery/sensor faults so the cyber layer is tested against physical anomalies rather than only clean-vs-attack data.
- **Reproducibility:** preserve code commit, environment file, dataset checksum, model hash, calibration/threshold version and final test report.

10. Verification and Validation Strategy

Test level	What to test	Typical method
Unit	parsers, timestamp rules, range/ rate rules, authorization policy, hashing	automated unit tests / boundary-value tests
Model offline	clean vs attack detection, unseen-device generalization	held-out datasets; confusion matrix; ROC/ PR
Component security	API auth, secrets, TLS, dependency vulnerabilities	SAST, SCA, secret scan, DAST/ API security tests
Integration	ingestion -> trust -> quarantine/ SIEM/ blockchain paths	service-integration tests with replay harness
Attack injection	spoof, replay, injection, DoS, credential misuse	controlled cyber range / test environment
Fault discrimination	sensor fault, hotspot, cooling fault, dropout	bench/ simulation fault injection
Performance	throughput, p95/ p99 latency, queue backpressure, CPU/ memory	load/ stress/ soak tests
Resilience	key vault unavailable, IDS timeout, SIEM unavailable, network partition	chaos/ failure-mode tests
Penetration	cloud/ API/ tenant isolation and privilege boundaries	independent penetration test / red team
Operational acceptance	SOC alert handling, ticketing, rollback, recovery	tabletop exercise + pilot shadow mode

11. Acceptance Criteria - Project Targets

The values below are proposed project acceptance targets. They are not universal regulatory limits. Final targets shall be reviewed and approved by the OEM/fleet product owner and aligned with the telemetry cadence, threat model, safety concept and deployment architecture.

ID	Acceptance item	Acceptance target	Evidence
ACC-01	Unauthorized device/ tenant	100% rejection in defined negative test set	auth test report
ACC-02	Replay/ freshness	100% detection of replay cases in DVP suite	replay injection report
ACC-03	Known attack detection	Recall >= 95% on agreed attack classes	held-out IDS test set
ACC-04	Clean-data false positives	FPR <= 1% on clean unseen sessions	clean replay campaign
ACC-05	Physics gross violations	100% detection of defined hard-range/ schema violations	boundary test report
ACC-06	Cyber vs physical-fault discrimination	No critical cyber classification for agreed genuine-fault baseline unless policy says otherwise	fault discrimination matrix
ACC-07	Trust decision latency	p95 cyber/ trust processing <= 50 ms for a 100 ms telemetry cadence, unless the approved system timing budget specifies a stricter value	load test
ACC-08	Quarantine effectiveness	BLOCK data never reaches trusted thermal feature pipeline	end-to-end trace test
ACC-09	Alert propagation	critical event visible to SIEM/ SOC within 5 s, unless the approved incident-response SLA specifies a stricter value	incident-flow test
ACC-10	Audit completeness	100% critical events contain required identity, time, version, reason, evidence hash	audit completeness script
ACC-11	Hash verification	100% anchored test events independently verify against stored off-chain evidence	blockchain verification test
ACC-12	Tenant isolation	0 unauthorized cross-tenant reads/ writes in test suite and pen test	RBAC/ ABAC + pen-test report
ACC-13	Crypto configuration	approved TLS / at-rest encryption active; no plaintext secrets	config scan + secret scan
ACC-14	Recovery	service returns to known-good operation after injected dependency/ key/ IDS failure without silently trusting unvalidated data	resilience test
ACC-15	Vulnerability closure	no unresolved Critical/ High findings at release unless formally risk-accepted	vulnerability register

12. Fail-Safe and Degraded-Mode Behavior

A trust layer can fail in two dangerous ways: fail-open (untrusted data is accepted) or fail-closed so aggressively that legitimate fleet telemetry becomes unavailable. The correct policy depends on data criticality, but must always be explicit and testable.

Failure	Recommended behavior	Downstream status
IDS service timeout	do not silently mark SAFE; fall back to identity + deterministic physics policy, set degraded flag	DEGRADED / policy dependent
Physics engine unavailable	retain identity/ IDS checks; tag missing physics evidence	DEGRADED
Key vault unavailable	reject new auth requiring key operation; continue only pre-authorized sessions if policy permits	DEGRADED/ BLOCK
SIEM unavailable	buffer signed/ hashed audit events locally or durable queue; do not drop evidence	SAFE data may continue if trust checks pass
Blockchain unavailable	store off-chain event + pending anchor job; do not block real-time trust decision solely for anchoring outage	PENDING ANCHOR
Schema version unknown	quarantine until supported decoder/ schema is available	BLOCK
Excessive false positives	switch model threshold only through controlled configuration; never manually bypass tenant isolation/ auth	controlled degraded mode

13. Requirements Traceability and DVP Structure

Each cybersecurity requirement should map to design element, source code/service, unit test, integration test, attack/fault injection case and release evidence. A simple traceability structure is shown below.

Requirement	Design block	Verification	Acceptance evidence
CYB-REQ-001 Authenticate every TCU/ device	API gateway + PKI/ key vault	AUTH-NEG-001..n	certificate and auth test report
CYB-REQ-002 Detect replay/ stale telemetry	freshness validator	REP-001..n	replay injection report
CYB-REQ-003 Detect defined spoofing patterns	Transformer IDS + physics engine	IDS-SPF-001..n	model metrics + scenario test
CYB-REQ-004 Prevent untrusted data entering thermal AI	trust policy + quarantine queue	E2E-Q-001..n	data-lineage trace
CYB-REQ-005 Preserve incident evidence	SIEM/ audit + hash anchor	AUD-001..n	audit completeness + hash verify
CYB-REQ-006 Enforce tenant isolation	authZ/ RBAC/ ABAC	TEN-001..n	negative API + pen-test result

14. Acceptance Gates and Sign-Off

Gate	Review question	Required evidence
G0 - Scope Freeze	Are assets, interfaces, tenants and trust boundaries defined?	architecture, asset list, data-flow diagram
G1 - Cyber Concept	Are threats, goals and fail-safe policies defined?	TARA/ threat model, cybersecurity requirements
G2 - Design Freeze	Are PKI, IDS, physics, SIEM, quarantine and evidence flows fully specified?	design + interface specs, rule/ model plan
G3 - Component Ready	Do all services pass unit/ static/ security scans?	unit report, SAST/ SCA/ secret scan, model card
G4 - Integrated Validation	Do SAFE/ BLOCK branches work under attack/ fault/ replay/ load scenarios?	DVP and integration/ attack reports
G5 - Security Acceptance	Are model metrics, pen test, tenant isolation and vulnerabilities acceptable?	model V&V;, pen test, vulnerability closure
G6 - Pilot Acceptance	Does the SOC/ fleet workflow work with acceptable false positives and latency?	shadow-mode pilot, SOC acceptance
G7 - Production Release	Is rollback, monitoring, key/ model/ rule lifecycle and incident response ready?	runbooks, SBOM, release manifest, sign-offs

15. Required Engineering Deliverables

- **Cybersecurity Plan:** scope, roles, lifecycle, evidence strategy and release gates.
- **Threat Model / TARA:** assets, threats, attack paths, mitigations and residual risk.
- **Cybersecurity Requirements Specification:** functional, crypto, logging, timing, tenant isolation and incident requirements.
- **Interface and Data Contract:** trusted/untrusted payload schema, metadata, reason codes and versioning.
- **Physics Rule Catalogue:** formula, rationale, calibrations, limits, severity and test cases.
- **IDS Model Package:** training dataset manifest, model card, features, metrics, threshold, hash and rollback artifact.
- **PKI/Key Management Design:** issue/rotate/revoke/recovery lifecycle.
- **SIEM and Incident Playbooks:** alert taxonomy, dashboards, escalation and recovery.
- **DVP and Traceability Matrix:** requirement-to-test-to-evidence mapping.
- **Security Test Evidence:** attack/fault injection, SAST/SCA/DAST, load, resilience, pen-test and closure reports.
- **Production Runbooks:** monitoring, threshold/model rollout, rollback, certificate compromise and incident response.

Suggested team roles

Role	Main ownership
Cybersecurity architect	threat model, trust architecture, requirements, acceptance
Backend/ cloud engineer	gateway, authZ, queues, quarantine, audit APIs
ML/ IDS engineer	dataset, Transformer IDS, metrics, threshold, model governance
Battery/ controls engineer	physics plausibility rules and fault discrimination
DevSecOps/ SRE	KMS, secrets, CI/ CD security scans, monitoring, resilience
SOC/ security analyst	SIEM rules, incident triage, playbooks and operational acceptance
V&V; / penetration team	independent attack, API, tenant and resilience validation

16. Recommended Standards / Assurance Baseline

The source architecture does not specify standards. The following references are recommended for an automotive cloud program and should be tailored to the target market, OEM process and contractual scope.

Reference	How it supports this layer
ISO/ SAE 21434	vehicle cybersecurity lifecycle, threat analysis, cybersecurity concept, verification and evidence
UNECE R155	vehicle cybersecurity management and regulatory expectations where applicable
UNECE R156 / ISO 24089	software-update engineering and governance if OTA model/ rule/ software updates are used
ISO/ IEC 27001	cloud/ SaaS information-security management system controls
NIST Cybersecurity Framework	govern-identify-protect-detect-respond-recover operational structure
OWASP API Security guidance	backend API authorization, BOLA, authentication and abuse controls
Cloud provider security/ KMS best practices	key management, encryption, logging, network segmentation and resilience

17. 12-Week Prototype-to-Acceptance Plan

Week	Focus	Exit deliverable
1	scope, assets, data contract, threat workshop	scope + threat model draft
2	security requirements, trust states, attack/ fault catalogue	requirements + DVP skeleton
3	device auth, schema/ freshness validator, quarantine queues	secure ingestion prototype
4	physics plausibility rules + calibration framework	physics validator v1
5-6	IDS dataset + Transformer baseline + thresholding	IDS model v1 + metrics
7	trust fusion policy + reason codes + downstream metadata	SAFE/ SUSPICIOUS/ BLOCK integration
8	SIEM/ audit + incident workflow + hash/ evidence anchor	incident evidence path
9	unit/ integration/ negative auth/ tenant tests	component test closure
10	attack + battery fault injection campaign	scenario test report
11	load/ resilience/ pen-test + vulnerability remediation	security acceptance evidence
12	shadow-mode pilot, SOC walkthrough, final threshold tuning	pilot acceptance + release package

18. Reference Material, Recent Research and Open Validation Resources

The references below support implementation of the Transformer IDS, multimodal trust fusion, CAN-traffic feature engineering, blockchain evidence handling and validation using realistic open datasets. Publisher and project links are provided for direct follow-up.

18.1 Recent IEEE / Reputed Journal Papers

Reference	Main relevance to Step 3	Link
Jo & Kim, IEEE Access, 2024 - Intrusion Detection Using Transformer in Controller Area Network	Transformer-based CAN IDS; useful as the closest IEEE baseline for temporal + payload-aware intrusion detection.	IEEE DOI
Jeong et al., IEEE Transactions on Intelligent Transportation Systems, 2024 - In-Vehicle Network Intrusion Detection System Using CAN Frame-Aware Features	Strong feature-engineering and low-latency benchmark for CAN IDS; useful to compare ML complexity against explainable frame-aware features.	IEEE DOI
Le et al., Knowledge-Based Systems, 2024 - Multi-classification in-vehicle intrusion detection system using packet- and sequence-level characteristics from time-embedded transformer with autoencoder	Time-embedded Transformer + autoencoder using packet- and sequence-level evidence; directly relevant to the proposed AI-based Cyber IDS.	Publisher / DOI
Kang et al., Vehicular Communications, 2024 - CANival: A multimodal approach to intrusion detection on the vehicle CAN bus	Combines timing and signal evidence; useful reference for multimodal trust fusion and open-source reproduction.	Publisher / DOI
Lampe & Meng, Computers & Security, 2024 - can-train-and-test: A curated CAN dataset for automotive intrusion detection	Open benchmark explicitly designed for known/ unknown vehicle and attack generalization tests.	Publisher / DOI
Andreica et al., Scientific Reports, 2024 - Blockchain integration for in-vehicle CAN bus intrusion detection systems with ISO/ SAE 21434 compliant reporting	Relevant to attack-evidence hashing, blockchain anchoring and cybersecurity reporting in the proposed architecture.	Open-access paper
Sun et al., IEEE Internet of Things Journal, 2024 - CCID-CAN: Cross-Chain Intrusion Detection on CAN Bus for Autonomous Vehicles	IEEE reference combining CAN intrusion detection with blockchain/ cross-chain evidence concepts.	IEEE DOI

18.2 Open Datasets and Reproducible Code

Resource	Recommended use for implementation / validation	Link
ROAD - Real ORNL Automotive Dynamometer CAN Intrusion Dataset	Use realistic ambient driving plus fuzzing, fabrication and masquerade cases; evaluate false positives and stealthier attacks.	Zenodo
can-train-and-test	Use cross-vehicle and unseen-attack splits instead of only random train/ test splits.	Figshare
SynCAN (ETAS)	Use signal-level synthetic attack cases for deterministic replay, spoofing and suppression experiments.	GitHub
CANival source code	Reproduce timing + signal multimodal IDS and compare anomaly scores / FPR with the proposed Transformer + physics fusion.	GitHub
CAN-IDSs-on-the-ROAD	Reference implementation for ROAD labeling, preprocessing and several ML/ DL baselines.	GitHub
SocketCAN / can-utils	Create virtual CAN interfaces, capture candump traces and replay controlled traffic in a safe test environment.	GitHub
ICSim	Open CAN/ instrument-cluster simulator for controlled attack and training exercises before HIL or vehicle testing.	GitHub
Hyperledger Fabric	Implement permissioned event/ hash anchoring and independently verify stored evidence hashes.	Documentation

18.3 Standards and Security Guidance

Reference	Use in the project	Link
ISO/ SAE 21434:2021 - Road vehicles - Cybersecurity engineering	Lifecycle, TARA, cybersecurity concept, verification and evidence planning.	ISO
UN Regulation No. 155 - Cyber security and cyber security management system	Regulatory cybersecurity-management expectations where applicable to the target vehicle market.	UNECE
NIST Cybersecurity Framework 2.0	Govern, Identify, Protect, Detect, Respond and Recover structure for cloud/ SOC operational controls.	NIST
OWASP API Security Top 10 - 2023	API authorization, authentication, resource-consumption, misconfiguration and unsafe-consumption test guidance.	OWASP

19. Implementation Sequence

Implement the layer in this order so that each capability can be demonstrated and verified independently before the complete trust-fusion stack is enabled.

Order	Build	Verification milestone	Target timing
1	JSON/ MQTT telemetry replay tool	normal battery stream reaches backend	Week 1
2	timestamp + sequence + schema validator	replay and malformed payload are rejected	Weeks 1-2
3	battery physics rules	impossible voltage/ SOC/ temp combinations are flagged	Weeks 2-4
4	basic statistical anomaly detector	rate/ frequency/ payload anomalies are detected	Week 4
5	Transformer IDS	sequence-based attack classification / anomaly score	Weeks 5-6
6	trust fusion	same event produces explainable SAFE/ SUSPICIOUS/ BLOCK decision	Week 7
7	quarantine + SIEM alert	blocked stream cannot reach feature pipeline and alert appears	Week 8
8	event hashing + Hyperledger anchor	stored event hash can be independently verified	Week 8
9	fault discrimination	real hotspot/ cooling fault is retained as trusted physical event, not mislabelled cyber	Weeks 9-10
10	load + negative security testing	system meets latency, false-positive and isolation targets	Weeks 10-11
11	shadow-mode pilot + release review	SOC workflow and final thresholds accepted	Week 12

Core engineering principle: **Trust = identity + data integrity + behavioral evidence + battery physics + policy.** AI is one contributor, not the entire cybersecurity decision.

20. Production Readiness Checklist

- All telemetry sources have unique managed identities and tenant binding.
- Unsupported/unknown schemas are rejected or quarantined.
- Replay and stale-message rules are enabled and tested.
- Physics rules have documented battery/OEM calibrations and boundary tests.
- IDS model has a model card, held-out evaluation, threshold rationale and rollback version.
- SAFE/SUSPICIOUS/BLOCK policy is documented with reason codes.
- Untrusted data is technically prevented from entering trusted thermal AI pipelines.
- SIEM events contain identity, timestamps, scores, versions, reason codes and evidence hash.
- Critical cyber alerts trigger defined OEM/SOC notification and incident playbook.
- Raw telemetry is stored off-chain; hashes/proofs are anchored according to data governance.
- Tenant isolation and API authorization have independent negative/penetration testing.
- No critical/high vulnerabilities remain unresolved without formal risk acceptance.
- Load, soak and dependency-failure tests meet service SLOs.
- Certificate/model/rule rotation and rollback are exercised before production.
- Pilot shadow-mode false-positive rate and alert workload are accepted by SOC/fleet users.

End of guide